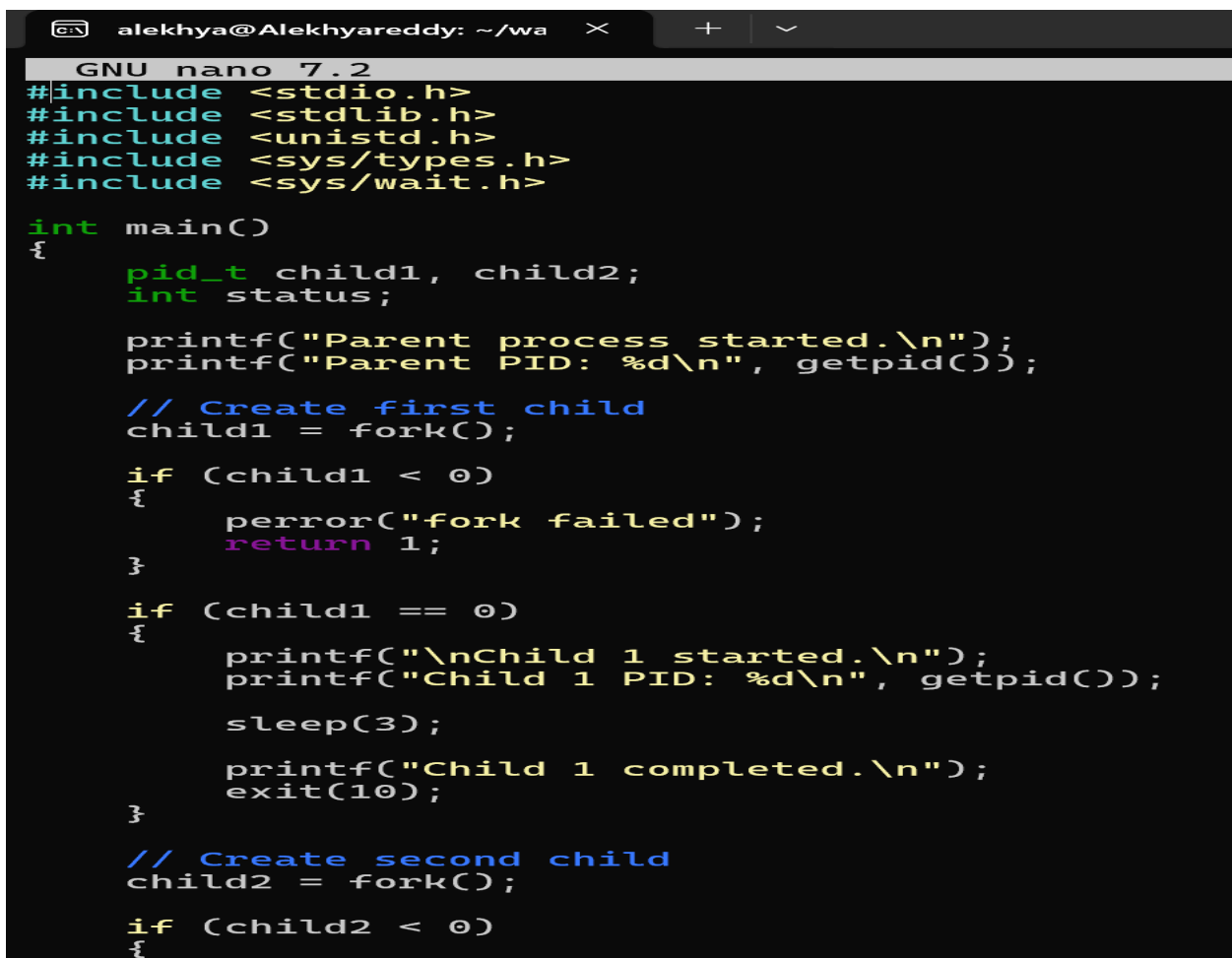


WEEK-6 SKILL

1 . Process Synchronization and Child Process Monitoring Using waitpid()

Aim : To create multiple child processes and use the **waitpid()** system call to synchronize and monitor child processes, obtain their termination status, and ensure that the parent process correctly waits for its children.

Program :

A screenshot of a terminal window with a dark background. The window title bar shows 'alekhya@AlekhyaReddy: ~/wa' and standard window controls. The terminal displays the GNU nano 7.2 editor interface. The code is a C program that creates two child processes using fork(). The parent process prints its own PID and then waits for the first child (child1) using waitpid(). After child1 completes, the parent prints a message and then creates a second child (child2). The code includes headers for stdio, stdlib, unistd, sys/types, and sys/wait. It uses printf for output, fork for process creation, perror for error handling, sleep for a delay, and exit for termination.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t child1, child2;
    int status;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());

    // Create first child
    child1 = fork();

    if (child1 < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child1 == 0)
    {
        printf("\nChild 1 started.\n");
        printf("Child 1 PID: %d\n", getpid());

        sleep(3);

        printf("Child 1 completed.\n");
        exit(10);
    }

    // Create second child
    child2 = fork();

    if (child2 < 0)
    {
```



alekhya@Alekhyareddy: ~/wa



GNU nano 7.2

```
if (child2 == 0)
{
    printf("\nChild 2 started.\n");
    printf("Child 2 PID: %d\n", getpid());

    sleep(5);

    printf("Child 2 completed.\n");
    exit(20);
}

// Parent waits for Child 1
printf("\nParent is waiting for Child 1...\n");

if (waitpid(child1, &status, 0) == -1)
{
    perror("waitpid failed");
    return 1;
}

if (WIFEXITED(status))
{
    printf("Child 1 terminated normally.\n");
    printf("Child 1 exit status: %d\n", WEXITSTATUS(status));
}

// Parent waits for Child 2
printf("\nParent is waiting for Child 2...\n");

if (waitpid(child2, &status, 0) == -1)
{
    perror("waitpid failed");
    return 1;
}

if (WIFEXITED(status))
{
    printf("Child 2 terminated normally.\n");
    printf("Child 2 exit status: %d\n", WEXITSTATUS(status));
}

printf("\nAll child processes have completed.\n");
printf("Parent process completed.\n");

return 0;
}
```

Output :

```
alekhya@Alekhyareddy: ~/wa × + v
alekhya@Alekhyareddy:~$ mkdir waitpid_practical
alekhya@Alekhyareddy:~$ cd waitpid_practical
alekhya@Alekhyareddy:~/waitpid_practical$ pwd
/home/alekhya/waitpid_practical
alekhya@Alekhyareddy:~/waitpid_practical$ nano waitpid_demo.c
alekhya@Alekhyareddy:~/waitpid_practical$ gcc waitpid_demo.c -o waitpid_demo
alekhya@Alekhyareddy:~/waitpid_practical$ ls
waitpid_demo  waitpid_demo.c
alekhya@Alekhyareddy:~/waitpid_practical$ ./waitpid_demo
Parent process started.
Parent PID: 625

Child 1 started.
Child 1 PID: 626

Parent is waiting for Child 1...

Child 2 started.
Child 2 PID: 627
Child 1 completed.
Child 1 terminated normally.
Child 1 exit status: 10

Parent is waiting for Child 2...
Child 2 completed.
Child 2 terminated normally.
Child 2 exit status: 20

All child processes have completed.
Parent process completed.
alekhya@Alekhyareddy:~/waitpid_practical$
```

2 . To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

Aim : To understand how Linux uses the **PATH** environment variable to search for executable commands, verify executable permissions, handle missing commands, and test command resolution.

Program :

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

int main()
{
    char *path;
    char *path_copy;
    char *dir;
    char full_path[1024];
    struct stat file_info;

    printf("==== PATH VARIABLE AND COMMAND RESOLUTION ====\\n\\n");

    /* Retrieve PATH variable */
    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable is not set.\\n");
        return 1;
    }

    printf("PATH variable:\\n%s\\n\\n", path);

    /* Make a copy because strtok modifies the string */
    path_copy = strdup(path);

    if (path_copy == NULL)
    {
        printf("Memory allocation failed.\\n");
        return 1;
    }

    printf("Search directories:\\n");

    dir = strtok(path_copy, ":");
```

```
while (dir != NULL)
{
    printf("%s\n", dir);
    dir = strtok(NULL, ":");
}

free(path_copy);

/* Test command resolution */
printf("\n==== COMMAND RESOLUTION =====\n");

const char *command = "ls";

path_copy = strdup(path);
dir = strtok(path_copy, ":");

while (dir != NULL)
{
    snprintf(full_path, sizeof(full_path), "%s/%s", dir, command);

    if (stat(full_path, &file_info) == 0)
    {
        printf("Command '%s' found at: %s\n", command, full_path);

        if (file_info.st_mode & S_IXUSR)
        {
            printf("Executable permission: YES\n");
        }
        else
        {
            printf("Executable permission: NO\n");
        }

        break;
    }

    dir = strtok(NULL, ":");
}

if (dir == NULL)
{
    printf("Command '%s' not found in PATH.\n", command);
}

free(path_copy);

/* Test missing command */
command = "abcxyz";

printf("\nTesting missing command: %s\n", command);

path_copy = strdup(path);
```



alekhya@Alekhyareddy: ~/pa



GNU nano 7.2

```
    dir = strtok(NULL, ":");
}

if (dir == NULL)
{
    printf("Command '%s' not found in PATH.\n", command);
}

free(path_copy);

/* Test missing command */
command = "abcxyz";

printf("\nTesting missing command: %s\n", command);

path_copy = strdup(path);
dir = strtok(path_copy, ":");

int found = 0;

while (dir != NULL)
{
    snprintf(full_path, sizeof(full_path), "%s/%s", dir, command);

    if (access(full_path, X_OK) == 0)
    {
        printf("Command found at: %s\n", full_path);
        found = 1;
        break;
    }

    dir = strtok(NULL, ":");
}

if (!found)
{
    printf("Command '%s' not found in PATH.\n", command);
}

free(path_copy);

printf("\n==== TEST COMPLETED =====\n");

return 0;
}
```

Output :

```
alekhya@AlekhyaReddy: ~/pa x + v
alekhya@AlekhyaReddy:~$ mkdir path_practical
alekhya@AlekhyaReddy:~$ cd path_practical
alekhya@AlekhyaReddy:~/path_practical$ pwd
/home/alekhya/path_practical
alekhya@AlekhyaReddy:~/path_practical$ nano path_resolution.c
alekhya@AlekhyaReddy:~/path_practical$ gcc path_resolution.c -o path_resolution
alekhya@AlekhyaReddy:~/path_practical$ ls
path_resolution  path_resolution.c
alekhya@AlekhyaReddy:~/path_practical$ ./path_resolution
===== PATH VARIABLE AND COMMAND RESOLUTION =====
PATH variable:
/home/alekhya/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/windows/system32:/mnt/c/windows:/mnt/c/windows/System32/Wbem:/mnt/c/windows/System32/WindowsPowerShell/v1.0:/mnt/c/windows/System32/OpenSSH:/mnt/c/Program Files/dotnet:/mnt/c/Program Files/nodejs:/mnt/c/Users/aleky/AppData/Local/Programs/Python/Launcher:/mnt/c/Users/aleky/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/aleky/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/aleky/AppData/Roaming/npm:/mnt/c/Program Files (x86)/MongoDB Atlas CLI:/mnt/c/Users/aleky/AppData/Local/Programs/mongosh:/snap/bin

Search directories:
/home/alekhya/bin
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/usr/lib/wsl/lib
/mnt/c/Program Files/Common Files/Oracle/Java/javapath
/mnt/c/windows/system32
/mnt/c/windows
/mnt/c/windows/System32/Wbem
/mnt/c/windows/System32/WindowsPowerShell/v1.0/
/mnt/c/windows/System32/OpenSSH/
/mnt/c/Program Files/dotnet/
/mnt/c/Program Files/nodejs/
/mnt/c/Users/aleky/AppData/Local/Programs/Python/Launcher/
/mnt/c/Users/aleky/AppData/Local/Microsoft/WindowsApps
/mnt/c/Users/aleky/AppData/Local/Programs/Microsoft VS Code/bin
/mnt/c/Users/aleky/AppData/Roaming/npm
/mnt/c/Program Files (x86)/MongoDB Atlas CLI/
/mnt/c/Users/aleky/AppData/Local/Programs/mongosh/
/snap/bin

===== COMMAND RESOLUTION =====
Command 'ls' found at: /usr/bin/ls
Executable permission: YES

Testing missing command: abcxzy
Command 'abcxyz' not found in PATH.
```

===== TEST COMPLETED =====

```
alekhya@AlekhyaReddy:~/path_practical$ which ls
/usr/bin/ls
alekhya@AlekhyaReddy:~/path_practical$ ls -l "$(which ls)"
-rwxr-xr-x 1 root root 142312 Jan 23 2026 /usr/bin/ls
alekhya@AlekhyaReddy:~/path_practical$ command -v ls
alias ls='ls --color=auto'
alekhya@AlekhyaReddy:~/path_practical$ command -v pwd
pwd
alekhya@AlekhyaReddy:~/path_practical$ command -v gcc
/usr/bin/gcc
alekhya@AlekhyaReddy:~/path_practical$ command -v abcxzy
alekhya@AlekhyaReddy:~/path_practical$ echo $?
1
```